

Rapport Final d'Optimisation Architecturale — Santana

Auteur : Claude Code, arbitrage final entre les rapports OpenClaw / Hermès / Claw **Date :** 20 juin 2026 **Méthode :** chaque affirmation des 3 rapports a été re-vérifiée directement sur le dépôt (`wc -l`, `grep`, `systemctl`, `ss`, `git blame`, `git show`, exécution complète de la suite de tests — 2 runs indépendants). Les chiffres ci-dessous sont mesurés à l'instant présent, pas recopiés des rapports précédents.

1. Objectif

Réduire Santana de ~55 200 lignes à ~23 000 lignes de **code technique**, sans perte fonctionnelle, sans régression, sans dépassement du budget BTR (10 €/mois). Zéro Redis, zéro API d'embeddings, zéro nouveau service — conformément à la règle de frugalité absolue du `CLAUDE.md` global.

Verdict sur la cible : atteignable et même dépassée, mais seulement si on l'applique au **code + données techniques** (Python + JSON + Shell), pas au dépôt brut. Les trois rapports convergent sur ce point et j'ai vérifié les chiffres qui le justifient (§2). La mémoire personnelle de Serge (12 043 lignes) reste hors périmètre.

2. État des lieux vérifié (mesures réelles, 20 juin 2026, 14h)

Catégorie	Mesure réelle	Commande
Python (tracké)	19 349 lignes , 97 fichiers existants	<code>git ls-files '*.py' \ xargs wc -l</code>
JSON (tracké)	15 337 lignes	<code>git ls-files '*.json' \ xargs wc -l</code>
	19 930 lignes	

Catégorie	Mesure réelle	Commande
Markdown (tracké)		<code>git ls-files -z '*.md' \ xargs -0 wc -l</code>
Shell (tracké)	928 lignes	<code>git ls-files '*.sh' \ xargs wc -l</code>
Code technique (Py+JSON+Sh)	35 614 lignes	somme
Mémoire personnelle protégée	12 043 lignes (livres 5 334 + archive 4 537 + backup_avant_reset 2 172)	<code>wc -l memory/livres/*.md memory/archive/*.md memory/backup_avant_reset/*.md</code>
Tests collectés	347	<code>pytest --collect-only -q</code>
Tests — baseline réelle	344 passed, 3 skipped, 0 failed (2 runs indépendants, 4min46)	<code>pytest -q</code>

Correction importante par rapport aux 3 rapports : Hermès et Claw rapportent tous les deux "341 passed, 3 failed" à cause d'un "bug atlas préexistant (748 vs 747 vecteurs)". **Ce bug n'existe pas dans l'état actuel** — `pytest tests/test_100_memoire.py::test_atlas_works` passe, et `memory/livres_index.json / memory/livres_embeddings.npy` sont synchronisés à 750/750. La suite complète confirme : **0 échec**. La vraie baseline est donc 344/347 (3 skip, pas liés à un bug), pas 341/347. Soit les deux rapports ont mesuré pendant une fenêtre où l'index était temporairement désynchronisé (le pipeline `atlas_engine/` tourne en tâche de fond), soit l'un a recopié l'autre sans réexécuter. **Leçon directement applicable à ce dépôt : ne jamais recopier un résultat de test sans le ré-exécuter.**

Le JSON mesuré (15 337) est légèrement supérieur aux 15 016 rapportés par Hermès/Claw — explication vérifiée : `souverainete_live.json` a grossi de 4 487 lignes **depuis le dernier commit, pendant cette seule session** (`git diff --stat souverainete_live.json` → "4487 insertions(+)"). C'est une preuve empirique supplémentaire, et plus forte que celle des 3 rapports, du problème décrit en §4.1.

3. Arbitrage des 3 rapports

3.1 Ce qui est confirmé, sans nuance

Proposition	Source	Vérification
Redis rejeté, SQLite (<code>metrics.db</code> , déjà présent, déjà WAL) suffit	Hermès, Claw	Confirmé. <code>metrics.db</code> existe (147 Ko, tables <code>tool_calls</code> / <code>errors</code> / <code>improvements</code>), <code>memory.db</code> (racine, pas <code>memory/memory.db</code>) tourne en WAL (<code>PRAGMA journal_mode → wal</code> , fichiers <code>-wal</code> / <code>-shm</code> présents). Aucune techno à ajouter.
Provider LLM unifié inutile	Hermès, Claw	Confirmé. <code>deepseek_client.py</code> est l'unique provider, règle BTR "jamais Claude API" tranche la question — pas un jugement architectural à faire, une règle déjà écrite dans <code>CLAUDE.md</code> global.
Architecture événementielle (Queue+Worker) inutile	Hermès, Claw	Confirmé. <code>asyncio.gather()</code> déjà en place (<code>core/react_loop.py:566</code> , commit <code>a4c82841</code> vérifié : "perf: parallélisation des outils search/memory").
Décorateur <code>@tool</code> unique déjà fait à 90%	Hermès, Claw	Confirmé. <code>ls tools/*_ops.py →</code> seul <code>memory_ops.py</code> reste. <code>tools/tools.py</code> fait 621 lignes, <code>tools/tools.json</code> ne référence plus que 21 outils .
Embeddings API LLM rejeté	Hermès, Claw	Confirmé, et de toute façon non négociable : <code>CLAUDE.md</code> global liste <code>sentence-transformers all-MiniLM-L6-v2</code> comme techno fixée, pas un point à arbitrer par coût.
<code>souverainete_live.json</code> = le vrai goulot de performance	Hermès (empirique), Claw (causal)	Confirmé et renforcé (§2, croissance de +4 487 lignes mesurée en direct). <code>agent/souverainete.py:529-618</code> fait <code>json.load + json.dump(indent=2)</code> complet sous <code>threading.Lock()</code> à chaque appel DeepSeek (<code>deepseek_client.py:21-22</code>) et chaque <code>web_search</code> (<code>tools/web_search.py:25-26</code>). Ring-buffer plafonné à 2 000 entrées

Proposition	Source	Vérification
		(MAX_LIVE_ENTRIES , actuellement 1 569) — donc pas une fuite, un problème de format de stockage, pas de volume de données.
7 autres fichiers JSON ont le même défaut (réécriture complète à chaque mutation)	Claw	Confirmé pour eval_history.json (agent/evaluator.py:181-203 , _load_history / _save_history rechargent et réécrivent tout). Pattern identique vérifié dans les 6 autres modules cités.
routes/codex_webapp.py et routes/files.py étaient bien morts	Hermès, Claw	Confirmé — déjà supprimés dans le working tree, api.py ne les importe plus (git diff api.py).
Bug crontab duplication knowledge/*.md	Hermès, Claw	Confirmé : crontab -l → job quotidien 4h30 qui copie ia.md / psychologie.md avec un suffixe date, sans purge. 14 doublons réels (29 mai → 20 juin). Correction nécessaire au-delà de la suppression des doublons : il faut aussi corriger/supprimer l'entrée crontab elle-même , sinon les doublons reviennent le lendemain — aucun des 3 rapports ne le formule aussi explicitement.
6 rapports d'audit racine, lignes citées	Claw	Vérifié ligne par ligne, exact : 130+150+224+139+133+465+143 = 1 384 lignes (le CLAUDE-CLOSURE.md à garder, référencé activement par CLAUDE.md).

3.2 Contradiction tranchée — routes/ Flask : Hermès avait raison, Claw a fait une erreur factuelle

Claw affirme (§1.5 de son rapport) : "santana-api.service (systemd) est actuellement actif" et recommande de ne pas toucher aux 5 fichiers routes/*.py restants pour cette raison.

Vérification directe (3 contrôles indépendants) :

```
systemctl --user is-active santana-api    → inactive
systemctl --user is-enabled santana-api   → disabled
```

```
systemctl --user show santana-api --property=ActiveEnterTimestamp → (vide)
ss -ltnp | grep 5000 → aucun résultat (api.py: app.run(port=5000))
```

Verdict : Claw s'est trompé sur ce point précis. Le service est bien `disabled + inactive`, jamais démarré dans cette session système, et **aucun processus n'écoute sur le port 5000**. Hermès avait raison : ce service est orphelin depuis le 31 mai.

Cela ne change pas la recommandation finale (ne pas supprimer sans accord explicite de Serge — un dashboard/Mini App peut être démarré manuellement à l'usage, et la convention du projet pour ce genre de suppression est l'accord écrit), mais le rapport de Claw présentait comme un fait vérifié quelque chose qui ne l'était pas. À retenir : **un "verdict mesuré" doit être réexécuté, pas seulement formulé avec assurance.**

3.3 Contradiction tranchée — analyse fonction-par-fonction de `agent/orchestration.py` : Claw a sous-estimé le code mort

Claw recommande de garder `health_check`, `delegate_to`, `record_tool_result`, `get_failure_status` (654 → ~430 lignes).

Vérification par grep des appelants réels de chaque fonction, hors fichier et hors tests :

Fonction	Appelants externes réels	Statut
<code>record_tool_result</code>	<code>tools/tools.py</code>	Vivante
<code>get_failure_status</code>	<code>tools/guardian.py</code>	Vivante
<code>health_check</code>	uniquement <code>agent/multi_agent.py</code> (lui-même 100% mort, voir §3.4)	Morte (transitivement)
<code>delegate_to</code>	uniquement <code>agent/multi_agent.py</code>	Morte (transitivement)
<code>detect_propagation</code> , <code>suggest_specialist</code> , <code>get_delegations</code> , <code>complete_delegation</code>	aucun	Mortes (confirmé, comme Claw)

Fonctions vivantes non identifiées par aucun des 3 rapports : `tenter_reparation()` (l.383-466, 84 lignes) et son helper `_notifier_serge()` (l.361-382, 22 lignes) sont appelés depuis `record_tool_result()` (l.507) — donc vivants, à conserver avec leurs dépendances `_load_failures / _save_failures` (l.339-360).

Footprint réel à conserver dans `orchestration.py` :
`_load_failures / _save_failures + _notifier_serge + tenter_reparation + record_tool_result + get_failure_status` $\approx$ **lignes 339-539, soit ~200 lignes (+ en-tête/imports nécessaires)**. Tout le reste (`health_check`, `detect_propagation`, `suggest_specialist`, `delegate_to`, `get_delegations`, `complete_delegation`, `_load_delegations / _save_delegations`) est mort, soit **~450 lignes mortes sur 654**, plus que les ~225 lignes estimées par Claw. `delegations.json` (48 lignes) devient intégralement orphelin.

3.4 Confirmé — `agent/decision.py` et `agent/multi_agent.py` 100% morts

```
grep -rn 'agent\.decision\b' --include='*.py' . | grep -v test → 0 résultat
grep -rn 'agent\.multi_agent\b|multi_agent_delegate\|multi_agent_health\|multi_agent_rou'
```

Confirmé par les 3 rapports, et vérifié une nouvelle fois : la seule référence à `agent.decision` dans un test (`test_guardian_consults_decision_and_patterns`, `tests/test_100_autonomie.py:69`) accepte `"agent.decision" in src` OR `"agent.patterns" in src` — et `tools/guardian.py:110` contient bien `from agent.patterns import detect_routines`. **Le test passe déjà sans `agent/decision.py`** — suppression sans risque pour ce test.

3.5 Confirmé et affiné — `agent/patterns.py`, `agent/securite.py`, `agent/souverainete.py`, `agent/proactive.py`, `agent/tracabilite.py`

Vérification fonction par fonction (grep des appelants hors fichier et hors tests) :

Fichier	Vivantes (appelant externe vérifié)	Mortes (0 appelant)
<code>agent/patterns.py</code> (380 l.)	<code>record_interaction</code> (<code>core/react_loop.py</code> , <code>tests</code>),	<code>record_preference</code> , <code>get_user_model</code> , <code>record_correction</code> ,

Fichier	Vivantes (appellant externe vérifié)	Mortes (0 appellant)
	<code>detect_routines</code> (<code>tools/guardian.py</code>)	<code>get_corrections</code> — et <code>detect_anomalie</code> sous condition, voir §4.2
<code>agent/securite.py</code> (348 l.)	<code>check_tool_rate_limit</code> (<code>tools/tools.py:573-575</code> , pré-condition avant dispatch , confirmé)	<code>get_rate_limit_status</code> , <code>set_tool_limits</code> , <code>get_all_tool_limits_status</code> , <code>get_access_logs</code> , <code>get_audit_stats</code>
<code>agent/souverainete.py</code> (633 l.)	<code>surveiller_appel_externe</code> (<code>deepseek_client.py:21</code> , <code>tools/web_search.py:25</code>)	<code>generate_report</code> , <code>get_latest_report</code> , <code>get_audit_summary</code> , <code>get_live_audit</code> , <code>audit_flow</code> , <code>_categorize_data_sources</code> confirmé mort (Claw l'avait signalé comme "à vérifier" — vérifié : 0 appellant)
<code>agent/proactive.py</code> (441 l.)	<code>evaluate_proactive_opportunity</code> (<code>tools/guardian.py</code>) + helpers internes (<code>can_suggest</code> , <code>is_good_timing</code> , <code>get_escalade_level</code> , <code>build_suggestion</code> , tous appelés en interne par la fonction vivante)	<code>get_proactive_stats</code>
<code>agent/tracabilite.py</code> (308 l.)	<code>log_action</code> (<code>core/react_loop.py</code> , <code>agent/orchestration.py</code> , <code>agent/souverainete.py</code> — plus large que rapporté , pas juste un appellant)	<code>log_tool_call</code> , <code>log_decision</code> , <code>log_response</code> , <code>log_error</code> , <code>export_logs</code> , <code>purge_old_logs</code> , <code>get_tracabilite_stats</code> , <code>get_recent_logs</code> , <code>clear_all_logs</code>

4. Découvertes non présentes dans les 3 rapports

4.1 Le commit cité par Hermès pour le "fix de continuation" est le mauvais commit

Hermès (§4.1) attribue au commit `908ee73b` la correction d'un bug de blocage "finish_reason=length → continuation → timeout 8 min". Claw reprend cette attribution sans la vérifier (§1.5, ligne "Boucle de continuation").

Vérification :

```
git show 908ee73b -- core/react_loop.py
```

Le diff réel de ce commit supprime un tout autre bloc : la **reformulation des messages longs** (>200 caractères, un appel DeepSeek supplémentaire avant analyse) — message de commit "rm: reformulation des messages longs (ralentissait sans gain)". Rien à voir avec `finish_reason="length"`.

Le vrai correctif ("Cas finish_reason='length' : livrer ce qu'on a, sans continuation", commentaire présent dans `core/react_loop.py` lignes 480-486) existe bien dans le code — mais `git blame` montre que ces lignes sont **"Not Committed Yet"** : c'est un changement non committé, qui traîne dans le working tree actuel, pas un fix livré par un commit identifiable.

Conséquence pratique : ce correctif doit être inclus explicitement dans le commit de la Phase 0 (§6), sous son propre message, pas supposé déjà acquis sous un hash existant.

4.2 Régression réelle déjà présente dans le nettoyage non committé — à corriger avant tout commit

```
tg_handlers/brainstorm.py:42 : from tools.pattern_ops import pattern_analyze
```

`tools/pattern_ops.py` est **déjà supprimé** dans le working tree. Cet import est dans un `try/except Exception ( _gather_friction() , l. 39-44)` — donc il ne plante rien, mais il **désactive silencieusement la détection de "conflit" en mode brainstorm** (une vraie fonctionnalité utilisateur : Phase 2 FRICTION du mode brainstorm, décrite dans l'en-tête du fichier).

Aucun test ne couvre ce chemin — c'est pourquoi 344/347 passent malgré cette régression.

Le correctif est mécanique : `tools/pattern_ops.py` (avant suppression, vérifié dans `git show HEAD:tools/pattern_ops.py`) ne faisait qu'appeler `agent.patterns.detect_anomalie(sujet=sujet)` et formater le résultat (`score_anomalie`, `raison`). Remplacer la ligne 42 par un appel direct à `from agent.patterns import detect_anomalie` et reconstruire `conflit` à partir du dict retourné (`raison` si `score_anomalie` dépasse un seuil).

Conséquence sur §3.5 : `detect_anomalie` n'est donc pas vraiment mort — il doit être **conservé**, avec le point d'appel corrigé. C'est la seule fonction du tableau §3.5 dont le statut change une fois ce correctif appliqué.

Ce point doit être traité avant de committer le travail de nettoyage déjà présent dans le working tree (Phase 0) — sinon le commit "Phase 0, testé 344/347" officialise une régression silencieuse.

5. Code mort confirmé — récapitulatif avec preuve

Fichier	Lignes	Action	Preuve (gr appelants externes)
<code>agent/decision.py</code>	257	Supprimer entièrement	0 résultat hors test qui accorde une alternative déjà vraie
<code>agent/multi_agent.py</code>	519	Supprimer entièrement	0 résultat ; entrées corréact_loop confirmées mortes (jamais dans <code>_TOOLS_BY_TOOL</code>)
<code>agent/orchestration.py</code>	654 → ~205	Garder <code>_load_failures / _save_failures / _notifier_serge / tenter_reparation /</code>	Voir §3.3, tableau d'appelants

Fichier	Lignes	Action	Preuve (gr appelants externes)
		<code>record_tool_result /</code> <code>get_failure_status</code> (l. 339-539) ; supprimer le reste (<code>health_check</code> , <code>detect_propagation</code> , <code>suggest_specialist</code> , <code>delegate_to</code> , <code>get_delegations</code> , <code>complete_delegation</code> , <code>_load_delegations /</code> <code>_save_delegations</code>)	
<code>agent/tracabilite.py</code>	308 → ~88	Garder <code>log_action</code> + rotation	Voir §3.5
<code>agent/patterns.py</code>	380 → ~245	Garder <code>record_interaction</code> , <code>detect_routines</code> , <code>detect_anomalie</code> (sous réserve du fix §4.2)	Voir §3.5 et
<code>agent/securite.py</code>	348 → ~218	Garder <code>check_tool_rate_limit</code> + helpers	Voir §3.5
<code>agent/souverainete.py</code>	633 → ~300 (avant migration SQLite)	Garder <code>surveiller_appel_externe</code> + <code>_load_live / _save_live</code> (migrés en SQLite Phase 2)	Voir §3.5
<code>agent/proactive.py</code>	441 → ~400	Supprimer seulement <code>get_proactive_stats</code>	Voir §3.5
<code>core/react_loop.py</code> (<code>_TOOL_CATEGORIES["orchestration"]</code>)	~6 (l. 88-93)	Supprimer l'entrée — 0 des 13 noms d'outils n'existe dans <code>tools.json</code> , jamais incluse dans <code>_TOOLS_BY_TYPE</code>	<code>grep -c '"<</code> <code>tools/tools</code> → 0 pour les
<code>delegations.json</code>	48	Supprimer (devient orphelin avec <code>get_delegations /</code> <code>complete_delegation</code>)	Plus aucun lecteur/écriv après nettoyy <code>orchestrati</code>
<code>tools/tools.py.bak.phase2</code>	— (non tracké,	Supprimer du disque	Fichier <code>.bak</code> hors git

Fichier	Lignes	Action	Preuve (gr appelants externes)
	déjà ignoré)		

⚠ **Hors périmètre, signalé mais non supprimé sans accord de Serge** : `atlas_engine/classifier.py::classify_message` contient du code mort confirmé (~40 lignes), mais `atlas_engine/` entier est une zone frozen (`CLAUDE.md`). `routes/*.py` (5 fichiers restants, ~600 lignes) : confirmé inactifs en pratique (§3.2) mais suppression nécessite l'accord explicite de Serge (convention du projet pour ce type de retrait).

Zones frozen vérifiées non touchées par ce plan : `agent/orchestrator.py` l.100-130 (contenu du prompt système — aucune modification ne s'en approche), `core/react_loop.py` l.242-245 (retour de `_run()` — le nettoyage `_TOOL_CATEGORIES` est aux lignes 88-93, à 150+ lignes de distance), `atlas_engine/` (aucune modification), `memory/memory.py` , `agent/context.py` (non concernés par ce plan).

6. Plan d'exécution final

Phase	Contenu	Durée réaliste	Test à chaque étape
Phase 0	1) Corriger <code>tg_handlers/brainstorm.py:42</code> (régression §4.2) avant tout commit. 2) Committer séparément le fix <code>finish_reason="length"</code> non attribué (§4.1) avec son propre message. 3) Committer les -3 207 lignes déjà retirées dans le working tree (<code>agent/decision_layer.py</code> , 9 wrappers <code>*_ops.py</code> , 2 routes mortes, <code>tools.json</code> réduit).	30 min	<code>pytest -q</code> → cible $\geq 344/347$, 0 échec (pas 341 — voir §2)
Phase 1	Supprimer <code>agent/decision.py</code> , <code>agent/multi_agent.py</code> ; réduire <code>orchestration.py</code> à ~205 lignes (§5, plus agressif que les 3 rapports), <code>tracabilite.py</code> , <code>patterns.py</code> ,	1 jour	<code>pytest -q</code> après chaque fichier

Phase	Contenu	Durée réaliste	Test à chaque étape
	securite.py , proactive.py aux fonctions vivantes confirmées ; nettoyer _TOOL_CATEGORIES["orchestration"] ; supprimer delegations.json et tools/tools.py.bak.phase2 . Tester après chaque fichier, pas en lot.		
Phase 2	Migrer les 7 fichiers JSON de logs (souverainete_live.json , eval_history.json , tracabilite_logs.json , tool_failures.json , pattern_data.json , securite_ratelimit.json , securite_audit.json) vers metrics.db (déjà présent, déjà WAL). souverainete_live.json est prioritaire (10 991 lignes, sur le chemin chaud de chaque appel DeepSeek/web_search — c'est la seule action de ce plan qui répond directement au problème de latence). Garder les JSON originaux renommés .json.migrated une semaine avant suppression définitive.	2-3 jours	Suite complète + script de migration testé sur copie d'abord
Phase 3	Corriger l'entrée crontab knowledge/*.md (pas seulement supprimer les 14 doublons — sinon ils reviennent le lendemain à 4h30) ; consolider les 6 rapports d'audit racine en un seul docs/archive/HISTORIQUE-AUDITS.md . Accord explicite de Serge requis avant suppression des audits (archives de décisions).	0,5 jour	Aucun test concerné (docs uniquement)
Phase 4	Suite complète des 347 tests (cible : 344+/347, 0 échec) ; benchmark latence avant/après sur 20 appels réels (core/santana_benchmark.py , mode quick) pour quantifier le gain de la Phase 2.	0,5 jour	Run complet, comparaison chiffrée
Total		4-5 jours, plusieurs	

Phase	Contenu	Durée réaliste	Test à chaque étape
		sessions (règle CLAUDE.md "un brief = une tâche précise")	

Backup complet (`cp -r ~/santana ~/santana-backup-$(date +%Y-%m-%d)`) avant Phase 1 et avant Phase 2 (règle globale #6) — **et nettoyage explicite après validation** (leçon du 19/06 : deux backups oubliés avaient fait monter le disque à 93%). Disque actuel vérifié : 77% utilisé (37G/48G, 12G disponibles) — marge suffisante pour ce plan, à surveiller si les backups s'accumulent.

7. Budget

Poste	Avant	Après ce plan
DeepSeek V4 Flash	Plafonné par <code>cost_governor.py</code>	Inchangé
Embeddings (MiniLM local)	0 €	0 € — confirmé, aucune migration API (et de toute façon non négociable, règle BTR globale)
Stockage (SQLite <code>metrics.db</code>)	0 € (déjà provisionné)	0 € — réutilisation, 0 nouvelle dépendance
Redis	Non utilisé	Rejeté, restera non utilisé
Total	—	+0 €/mois. Budget BTR 10 €/mois intact.

8. Risques et mitigations

Risque	Probabilité	Mitigation
Régression silencieuse non	Confirmée présente,	Grep systématique <code>getattr\ importlib\ __import__</code> avant

Risque	Probabilité	Mitigation
couverte par les tests (comme <code>brainstorm.py</code> , §4.2)	pas hypothétique	chaque suppression (a déjà révélé <code>agent/</code> <code>orchestration.py::health_check</code> qui fait <code>__import__(f"agent.{mod}")</code> sur une liste codée en dur incluant <code>decision_layer / decision</code> — heureusement cette fonction est elle-même morte, voir §3.3, donc sans conséquence une fois supprimée). Tester fichier par fichier, jamais en lot.
Perte de données en migration JSON→SQLite (Phase 2)	Moyenne	Script testé sur copie, JSON originaux gardés <code>.json.migrated</code> 1 semaine, backup complet avant la phase
Confusion baseline de tests	Résolue par ce rapport	Baseline documentée et réexécutée deux fois : 344/347, 0 échec — toute régression future se compare à ce chiffre, pas à 341
Suppression de routes/ par erreur	Nulle dans ce plan	Aucune suppression proposée — accord de Serge requis si décidé séparément (§3.2)
Crontab non corrigé en Phase 3	Faible si oublié	Le plan précise explicitement : corriger le job, pas seulement les fichiers (§6, Phase 3)
Zones frozen touchées	Faible	Vérifié par numéro de ligne (§5) : aucune modification proposée ne touche <code>orchestrator.py</code> l.100-130 ni <code>react_loop.py</code> l.242-245

9. Gains attendus

- **Lignes Python confirmées supprimables (Phase 1) :**
`agent/decision.py` (-257) + `agent/multi_agent.py` (-519) + réduction `orchestration.py` (-449, plus agressif que les 3 rapports) + `tracabilite.py` (-220) + `patterns.py` (-135, sous réserve §4.2) + `securite.py` (-130) + `souverainete.py` (-135 avant migration) + `proactive.py` (-40) + `_TOOL_CATEGORIES` (-6) ≈ **-1 890 lignes**, soit 19 349 → **~17 460**.

- **Lignes JSON (Phase 2) :** 15 337 → ~500 (migration SQLite de 7 fichiers, `souverainete_live.json` en tête).
- **Code technique total (Py+JSON+Sh) :** 35 614 → ~**18 890**, sous la barre des 23 000 visée, marge confortable.
- **Performance :** suppression du read-modify-write JSON synchrone sous verrou sur le chemin chaud de chaque appel DeepSeek/web_search — le seul levier de ce plan qui répond directement à la lenteur observée. (Note indépendante : deux autres bugs de latence déjà documentés et non corrigés — notification parasite et +2s sur les outils rapides via le heartbeat — restent hors périmètre de cette mission de réduction de lignes, voir mémoire `audit_optimisations_2026-06-20` ; à traiter séparément si Serge le souhaite.)
- **Fiabilité :** une régression réelle (§4.2) est identifiée et corrigée avant même le premier commit de ce plan — sans cette vérification, elle serait passée inaperçue indéfiniment (aucun test ne la couvre).
- **Maintenabilité :** fin de la confusion fichier mort/vivant pour 6 modules `agent/*`, vérifiée fonction par fonction plutôt que fichier par fichier ; un seul format de stockage (SQLite) pour les logs/état.
- **Tests :** baseline corrigée et vérifiée deux fois : 344/347, 0 échec (au lieu des 341/347 supposés par les 2 rapports précédents).